SparkScope Repair Estimator - Submission Writeup

Prepared by Dhyan | Spark Homes Developer Contest | June 2026

A field-first Progressive Web App for Spark Homes acquisition walkthroughs. Built as a static vanilla HTML/CSS/JavaScript submission with offline persistence, PWA assets, configurable room instances, photo capture, and ZIP/XLSX export.

1. Design / UX decision	The main design decision was to make the estimate room-instance based instead of only category based. Agents can add Bathroom 2, Bedroom 3, or a Living / Common Area while walking the property, and each instance gets clearly labeled groups such as Bathroom 2: Tub & Shower. That keeps the checklist aligned to the house in front of them and makes missed rooms easier to notice.
2. Broken or fragile	Photos are compressed and saved in localStorage, which keeps the app simple and fully offline but can hit browser quota on very large projects. Serial parsing is best-effort: it uses browser BarcodeDetector/TextDetector APIs when available and otherwise leaves the serial field editable. PWA install/offline caching requires serving the folder over http or https; direct file opening still runs the estimator but cannot register a service worker.
3. Creative addition	I added a Deal Analyzer that turns the repair estimate into acquisition guidance. Agents can enter ARV, purchase price, holding costs, selling costs, contingency, target profit, and square footage; the app calculates max offer, projected profit, total basis, and repair dollars per square foot. This makes the repair estimate immediately useful for offer decisions, not just internal scope tracking.
4. Next two days	I would move photo storage to IndexedDB for larger projects, add automated mobile regression tests for export/offline flows, tune the room templates with real Spark walkthrough feedback, and improve serial capture with a guided equipment-photo mode. I would also publish a hosted GitHub Pages version and add a tiny test fixture suite for price CSV imports.
AI tools	AI assistance was used to translate the brief into an implementation plan, draft the first version of the static app, and prepare this writeup. The final structure, required-feature mapping, price-list treatment, and verification pass were kept tied to the contest brief and provided CSV.

Submission note: SparkScope uses the provided pricing CSV as the default price source, includes all 108 listed repair items, and exports a ZIP with the Excel estimate plus project photos.